EcoGestor API no Desafio DevOps

Projeto Cidades ESG Inteligentes

Giovanni Luiz do Rego · RM 563071
FIAP · Fase 6 · 13 de outubro de 2026

Documento de preparação técnica
Código adaptado e validação local concluída. Execuções de Docker, pipeline remoto e deploys ainda pendentes. Este documento deve receber as evidências reais antes da entrega acadêmica.

Objetivo e continuidade do projeto

O EcoGestor apoia a gestão de resíduos e reciclagem por meio de unidades, pontos de coleta, coletas, destinações e alertas. A proposta tem foco ambiental e favorece a rastreabilidade das operações, alinhada ao tema ESG.

Esta fase parte do código C# do EcoGestor API encontrado no arquivo EcoGestorApi_CodigoFonte. A adaptação mantém SQL Server e Entity Framework Core. A atividade posterior com MongoDB é um trabalho distinto e seus prints não comprovam a execução deste pipeline.

Escopo implementado

Componente	Preparação realizada
Aplicação	ASP.NET Core 8; autenticação JWT; senha administrativa armazenada como hash; healthchecks.
Testes	12 testes aprovados localmente, incluindo acesso protegido e CRUD.
Docker	Dockerfile em múltiplas etapas e Compose com API, SQL Server, rede e volume.
CI/CD	GitHub Actions com build, testes, imagem, staging e produção sequenciais.
Documentação	README, guia GitHub, registro de validação e roteiro de evidências.

Os dois ambientes serão uma simulação acadêmica persistente no computador Windows, sem contratação de nuvem. A conclusão da atividade exige executar e documentar ambos.

Arquitetura e execução com Docker

Cada ambiente possui uma API e uma instância SQL Server separadas. Docker Compose atribui redes e volumes próprios a cada nome de projeto, evitando compartilhar os dados entre staging e produção.

Ambiente	Projeto Compose	Endereço local
Desenvolvimento	ecogestor-local	http://localhost:8080
Staging	ecogestor-staging	http://localhost:8081
Produção simulada	ecogestor-production	http://localhost:8082

Dockerfile e imagem

A etapa SDK restaura os pacotes e publica a API em Release. A etapa final utiliza a imagem ASP.NET 8, executa com o usuário app e verifica /health/ready com curl. O Dockerfile completo está na raiz e também no README.

Nome planejado: **ecogestor-api:<SHA do commit>**. A imagem aprovada é salva como artefato do GitHub e carregada no Docker local. Ela ainda não foi construída nesta validação.

Banco e configuração

SQL Server 2022 Developer é usado somente para demonstração acadêmica. Não há porta de banco publicada no computador. O volume sql-data é preservado nas atualizações. A API espera o banco ficar saudável, aplica a migration e cria o administrador se ainda não houver usuários.

DB_PASSWORD, JWT_KEY e ADMIN_PASSWORD são fornecidos por configuração externa. O arquivo .env.example não inclui senhas reais. Para uso público futuro, será necessário preparar HTTPS, credenciais com privilégio mínimo, backup e hospedagem apropriada.

Comandos de execução local

./scripts/New-LocalEnvironment.ps1

docker compose -p ecogestor-local up -d --build

docker compose -p ecogestor-local ps

docker compose -p ecogestor-local stop

O README apresenta a opção --wait e o tempo limite de inicialização. O script gera .env; consulte as credenciais nele para login sem colocá-las em prints.

As migrations desta adaptação são destinadas a bancos novos da fase DevOps. Não devem ser aplicadas diretamente sobre um banco antigo sem um plano específico de migração.

Pipeline de integração e deployment

Ferramenta escolhida: GitHub Actions. O workflow principal está em .github/workflows/ci-cd.yml. O deployment reutilizável está em deploy.yml.

Etapa	Execução e critério para avançar
1 Build	Runner Ubuntu do GitHub restaura dependências e compila a solução em Release.
2 Testes	Executa xUnit; falhas bloqueiam as etapas seguintes. Guarda os resultados TRX como artefato.
3 Imagem e integração	Constrói a imagem, inicia API + SQL Server com Compose e verifica prontidão e consulta de unidades.
4 Staging	Runner Windows carrega a imagem aprovada e atualiza ecogestor-staging. Confere ambiente, versão, dados, login e relatório.
5 Produção	Depende do sucesso de staging. Publica a mesma imagem em ecogestor-production e repete as verificações.

Disparos e controle

Pull requests para main executam apenas integração. Pushes e execuções manuais em main podem avançar aos deploys. Execuções concorrentes da mesma branch são enfileiradas. A promoção é automática, salvo se o usuário configurar uma aprovação no environment do GitHub.

Configuração externa ainda necessária

Criar o repositório, cadastrar environments staging e production, fornecer os três secrets por ambiente e registrar um runner Windows x64 com a etiqueta ecogestor. Docker Desktop deve usar Linux containers. O guia docs/CONFIGURACAO.md explica os passos.

Persistência e limitações

O runner deve operar na conta que tem acesso ao Docker Desktop. Os serviços permanecem no computador após o job, e não nos runners temporários do GitHub. Os endereços localhost só funcionam nesse computador. Não há rollback automático; uma reversão precisa considerar a versão da aplicação e o esquema do banco.

Validação e desafios encontrados

Validações executadas em 13 de outubro de 2026

Verificação	Resultado confirmado
Compilação da solução	Release, zero erros e zero avisos. SDK .NET 8.0.425.
Testes automatizados	12 aprovados, zero falhas, zero ignorados.
Configuração Compose	Aceita pelo comando config --quiet do Compose 5.6.0.
Workflows GitHub	Aceitos pelo actionlint 1.7.12.
Scripts PowerShell	Três scripts passaram pela análise sintática.

Os testes incluem login, senha inválida, recusa de acesso sem token, consulta de endpoints, CRUD de unidades, identificação do ambiente e geração da migration SQL. O detalhamento está em docs/VALIDACAO.md e docs/validacao-local.json.

Limite da validação: os testes HTTP usam banco InMemory. O teste de migration gera SQL sem conexão. A análise do Compose e dos workflows é estática. Docker Desktop não está instalado e não houve execução de SQL Server, containers ou pipeline remoto nesta preparação.

Desafios e soluções aplicadas

Problema observado	Ajuste realizado
Testes sem importação xUnit	Adicionada importação global para os atributos e interfaces de teste.
Configuração de banco dos testes	Separado o provedor InMemory do SQL Server, com um banco por fixture.
Migration original incompleta	Regenerados migration, metadados de descoberta e snapshot para novos bancos.
Senha fixa em texto simples	Administrador inicial configurável e armazenamento de hash de senha.
Inicialização JWT nos testes	Leitura da configuração durante criação das opções de autenticação.

Evidências e checklist da entrega

As evidências abaixo devem ser capturadas após as execuções reais. Inserir imagens legíveis com o comando ou etapa e seu resultado, sem senhas ou tokens. Registrar o link da execução e o SHA do commit.

Código	Evidência a inserir	Estado
E01	Build no GitHub Actions	Pendente
E02	Testes automatizados aprovados no pipeline	Pendente
E03	Imagem Docker e integração com SQL Server	Pendente
E04	Job de deployment em staging aprovado	Pendente
E05	Staging com ambiente, versão e dados	Pendente
E06	Job de deployment em produção aprovado	Pendente
E07	Produção com mesma versão e dados	Pendente
E08	Containers e volumes separados no Docker	Pendente

Checklist obrigatório

Item do enunciado	Situação atual
Projeto ZIP com estrutura organizada	Pacote de preparação; revisar novamente após as evidências.
Dockerfile funcional	Escrito; funcionamento em container ainda pendente.
Compose ou Kubernetes	Compose escrito e configuração validada. Execução pendente.
Pipeline com build, teste e deploy	Escrito e validado estaticamente. Execução remota pendente.
README com instruções e prints	Instruções prontas. Prints ainda pendentes.
PDF ou PPT com evidências	Este PDF é preparatório; inserir evidências reais.
Deploy em staging e produção	Ambos pendentes de execução.

Base técnica: enunciado Desafio DevOps, material FIAP da Fase 6, documentação oficial Docker Compose, SQL Server em containers e GitHub Actions. Links disponíveis no README.